Project Report

Online Payment System

Introduction

This report presents the design, implementation, and functionality of a web-based multi-user payment system developed using the Django framework. Inspired by online payment services like PayPal, this project enables users to securely register, manage their accounts, send payments, and request money from others. The system includes a robust authentication process, a RESTful currency conversion service, and an administrative interface for overseeing transactions. Additionally, the project ensures transactional integrity using Django's built-in ACID-compliant mechanisms, provides a secure communication layer through HTTPS, and integrates a timestamping service via Apache Thrift. The application has been successfully deployed on Amazon AWS, demonstrating its scalability and real-world usability. This report details the implementation of each layer, assessing the completeness of the system while also offering a user manual for seamless navigation.

1. Presentation Layer

The Presentation Layer consists of templates that enable user interactions with the web application.

Implementation Status: Fully Implemented

- Users can:
  - View all transactions.
  - Make direct payments to other registered users.
  - Request payments from other registered users.
- Administrators can:
  - View all user accounts and balances.
  - See all payment transactions.
  - Register new administrators.

Key Features

- **Templates:**
  - Includes `dashboard.html`, `register.html`, `login.html`, `admin_dashboard.html`, payment pages, and more.
- **Navigation Rules:**
  - URL routing ensures proper page navigation.

- Access restrictions implemented for user/admin pages.

2. Business Logic Layer

The Business Logic Layer contains views that manage transactions, payments, and database interactions while ensuring ACID compliance (Atomicity, Consistency, Isolation, Durability).

Implementation Status: Fully Implemented

- Users can:
  - View transaction history.
  - Make direct payments to other users.
  - Accept or reject payment requests.
- Administrators can:
  - View all accounts and balances.
  - Register additional administrators.

Key Features

- **Transaction Management:**
  - `direct_payment` view ensures correct currency conversion before transferring funds.
  - `accept_payment_request` view processes requests securely, ensuring accurate conversions.
- **ACID Compliance:**
  - All transactions are atomic (processed as a single unit).
  - Django's transaction system ensures consistency and data integrity.

3. Data Access Layer

The Data Access Layer consists of Django models that store user accounts, transactions, and payment requests.

Implementation Status: Fully Implemented (Using SQLite)

- `OnlineAccount` model tracks user balance and selected currency.
- `Payment` model logs all transactions, including sender, recipient, amount, and type.
- `PaymentRequest` model manages pending payment requests between users.

Key Features

- **Relational Integrity:**
  - Foreign key relationships ensure consistency between users and payments.
- **Optimized Querying:**
  - Efficient database operations using Django ORM methods (`get()` and `filter()`).

- **Migration Support:**
    - Database schema updates handled via Django migrations for seamless updates.

4. Security Layer

The **Security Layer** ensures authentication, authorization, and protection against web vulnerabilities.

Implementation Status: Fully Implemented

- **Authentication & Access Control:**
    - Users can register, log in, and log out securely.
    - Access is restricted based on roles (regular users vs. administrators).
- **HTTPS Enforcement:**
    - All interactions take place over HTTPS for secure communication.
- **Protection Against Web Threats:**
    - Cross-Site Scripting (XSS) prevention.
    - Cross-Site Request Forgery (CSRF) protection.
    - SQL injection mitigation using Django ORM.
    - Clickjacking protection via `X-Frame-Options`.

5. Web Services (RESTful API)

A **RESTful Web Service** is implemented for currency conversion.

Implementation Status: Fully Implemented

- **Endpoint:**
- `https://127.0.0.1:8000/payapp/conversion/{currency1}/{currency2}/{amount}/`
- **Functionality:**
    - Converts currency between GBP, USD, and EUR using **hardcoded rates**.
    - Accepts **GET requests** only.
- **Example Usage:**
    - Request:
    - `GET /payapp/conversion/USD/GBP/100/`
    - Response:

```
json
{ "converted_amount": 75.00, "rate": 0.75 }
```

6. RPC (Apache Thrift)

A **Thrift-based timestamping service** provides transaction timestamps.

Implementation Status: Fully Implemented

- **Thrift Server:**
  - Runs on **port 10000**, returning timestamps upon request.
- **Integration with Payments:**
  - All transactions receive timestamps when processed.

Example Usage

- Requesting a timestamp from the Thrift service:
- `thrift_client.get_timestamp()`
- Response:
- `"2025-04-05 18:30:01"`

7. Cloud Deployment (Amazon AWS)

The application has been successfully **deployed on AWS EC2**.

Implementation Status: Fully Implemented

- **Deployment Process:**
  - Hosted on **Amazon EC2** using Django.
  - Configured **Gunicorn & Nginx** for production.
  - Enabled **HTTPS encryption** for secure interactions.

* AWS deployment screenshots and terminal code are provided at the end of the report in AWS section.

User Manual

This section contains the graphics and instructions to use the Online Payment System web app.

1. Home Page

After initiating the development server web app can be accessed using URL https//: 127.0.0.1:8000(development server). Upon entering the URL user will be directed to the homepage of Online Payment System Website as shown in **Figure 1** below.

The **Home Page** serves as the welcoming interface of the payment system, offering users a clear entry point into the platform. Designed with a clean and intuitive layout, it provides easy navigation options for new users to register and existing users to log in.

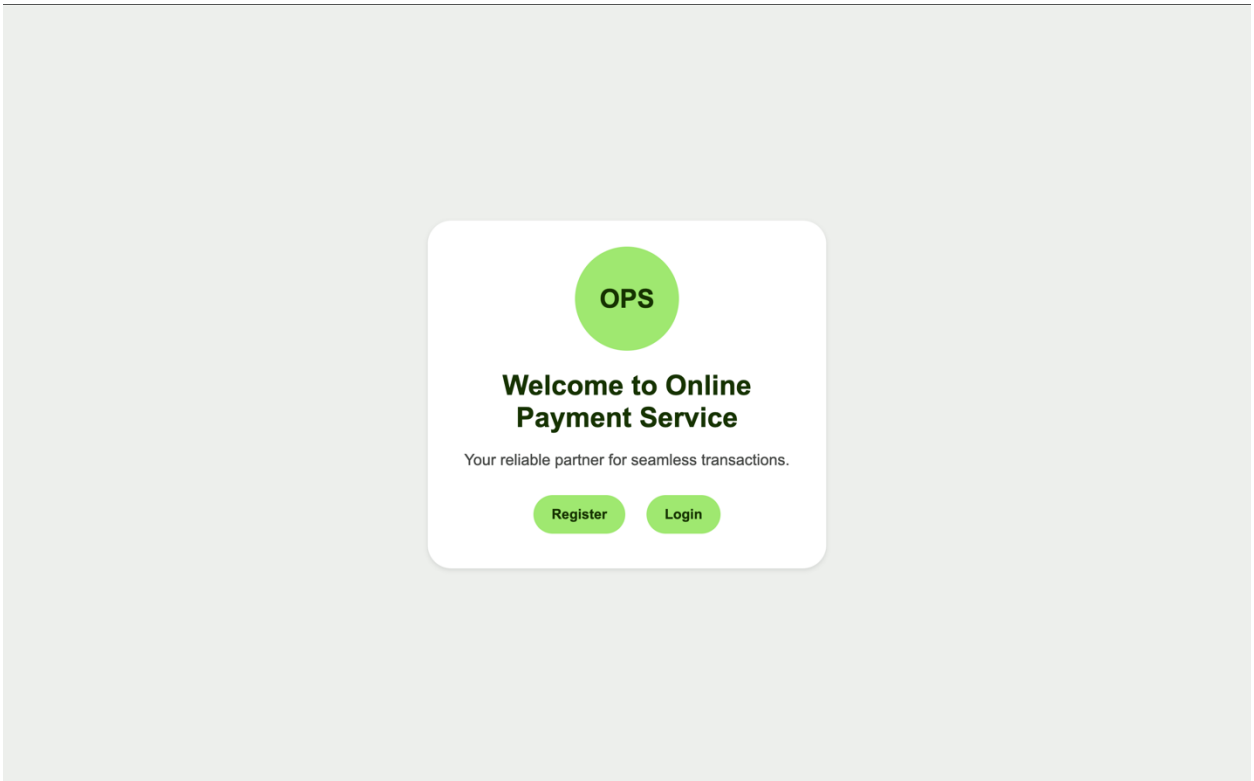


Figure 1. Home page of payment app.

Users can access registration and login links via well-placed buttons, ensuring a seamless onboarding experience. The page also maintains a responsive design**,** making it accessible across devices like desktops, tablets, and smartphones. This page establishes a professional and user-friendly foundation, allowing users to smoothly transition into the platform's functionalities.

2. User Registration

Figure 2 shows the user registration page which can be navigated to by clicking register button on the home page of the online payment system. The **User Registration** process allows new users to securely create an account and set up their payment preferences. To register, users need to provide a **username, email, password, and preferred currency (GBP, USD, or EUR)**. Upon submission, the system assigns an initial balance based on the selected currency and securely stores the credentials using Django's built-in authentication framework. The registration page features intuitive form validation to ensure users input valid details, preventing errors. Once registered, users can proceed to **log in** and access their **personal dashboard**, where they can initiate transactions and manage their account.

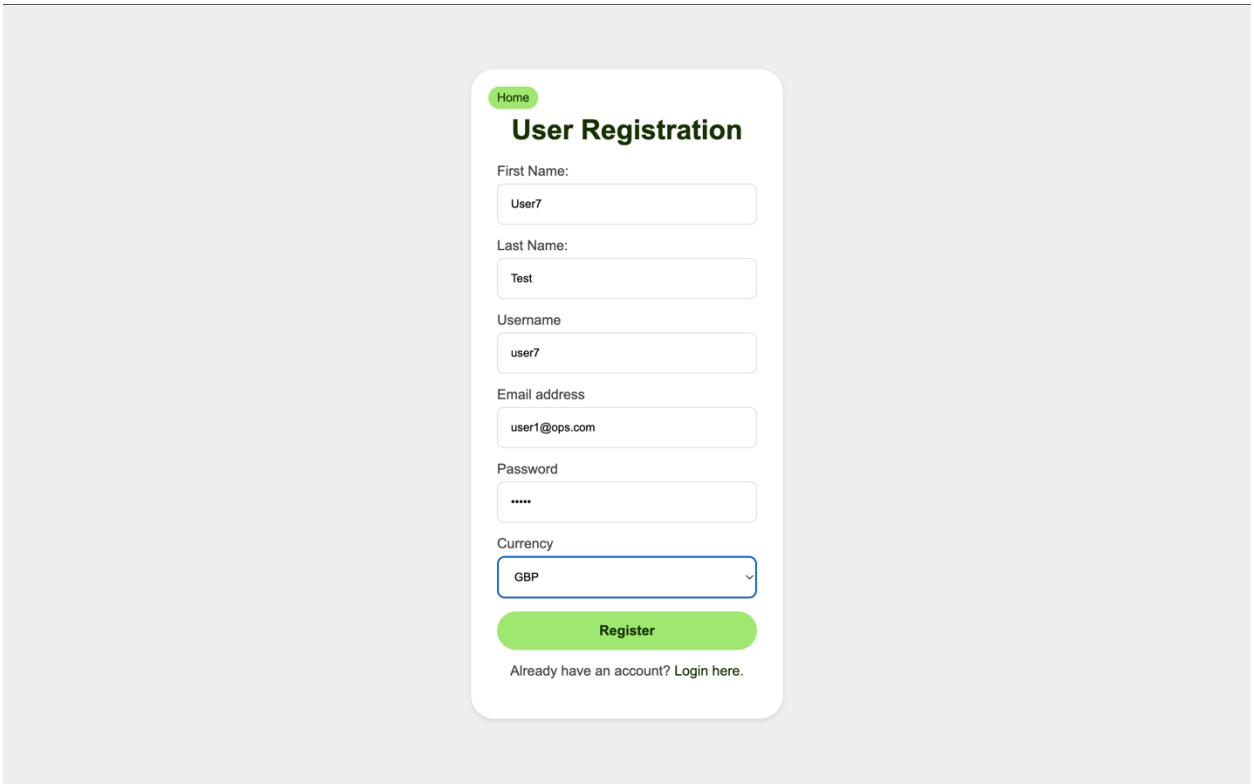


Figure 2. User registration page.

3. User Login

The **User Login** page (as seen in **Figure** 3) grants users' access to their personalized dashboard, where they can manage transactions. To log in, users must enter their **registered username and password**, which are securely verified against the database. If the credentials match, users are redirected to their **dashboard where** they can view their balance, send or request payments, and track transaction history. In case of incorrect login details, the system provides **error messages**, guiding users to check their credentials. Authentication security measures, including **password hashing**, ensure user data remains protected.

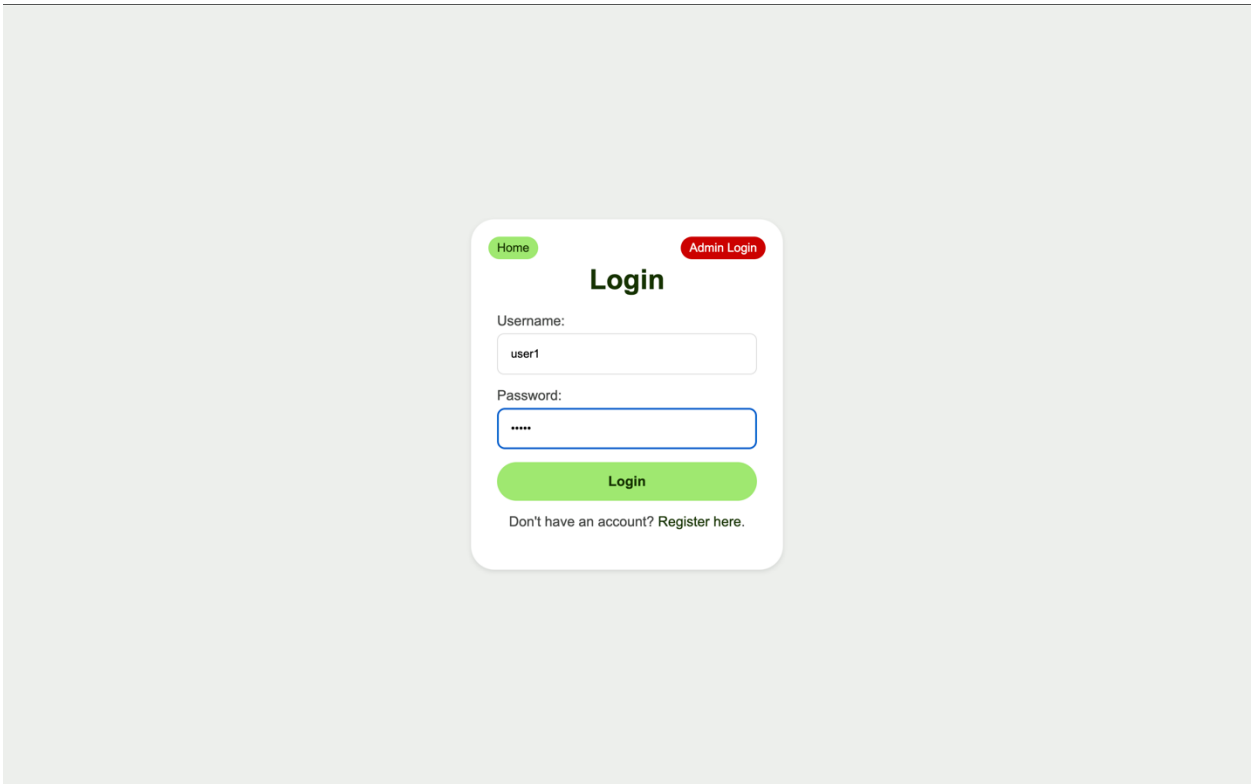


Figure 3. User login page.

4. Dashboard

The **User Dashboard** (as seen in **Figure 4)** is the central hub where registered users can **view and manage their account details, transactions, and financial activities**. Upon logging in, users are presented with an intuitive interface that displays their **current balance, recent payments, and payment requests**. The dashboard enables users to **send direct payments**, **request money from other users**, and **review past transactions**, ensuring complete control over their financial interactions. Additionally, key navigation buttons allow users to **access the RESTful currency conversion service** and securely manage their profile. Security measures are enforced, ensuring that users can **only see their own data**, maintaining privacy and protection. The dashboard layout is designed to be **user-friendly, responsive, and easy to navigate**, allowing seamless interaction with all payment functionalities.

At the **bottom of the dashboard**, users can access the **notification tabs**, which display important financial alerts. Notifications include **received payment requests, unread transactions, and successful payments**. Each notification is dynamically updated, ensuring users stay informed about their account activity. Users can **mark notifications as read** to streamline their dashboard view. This system ensures smooth communication and allows users to react promptly to financial interactions.

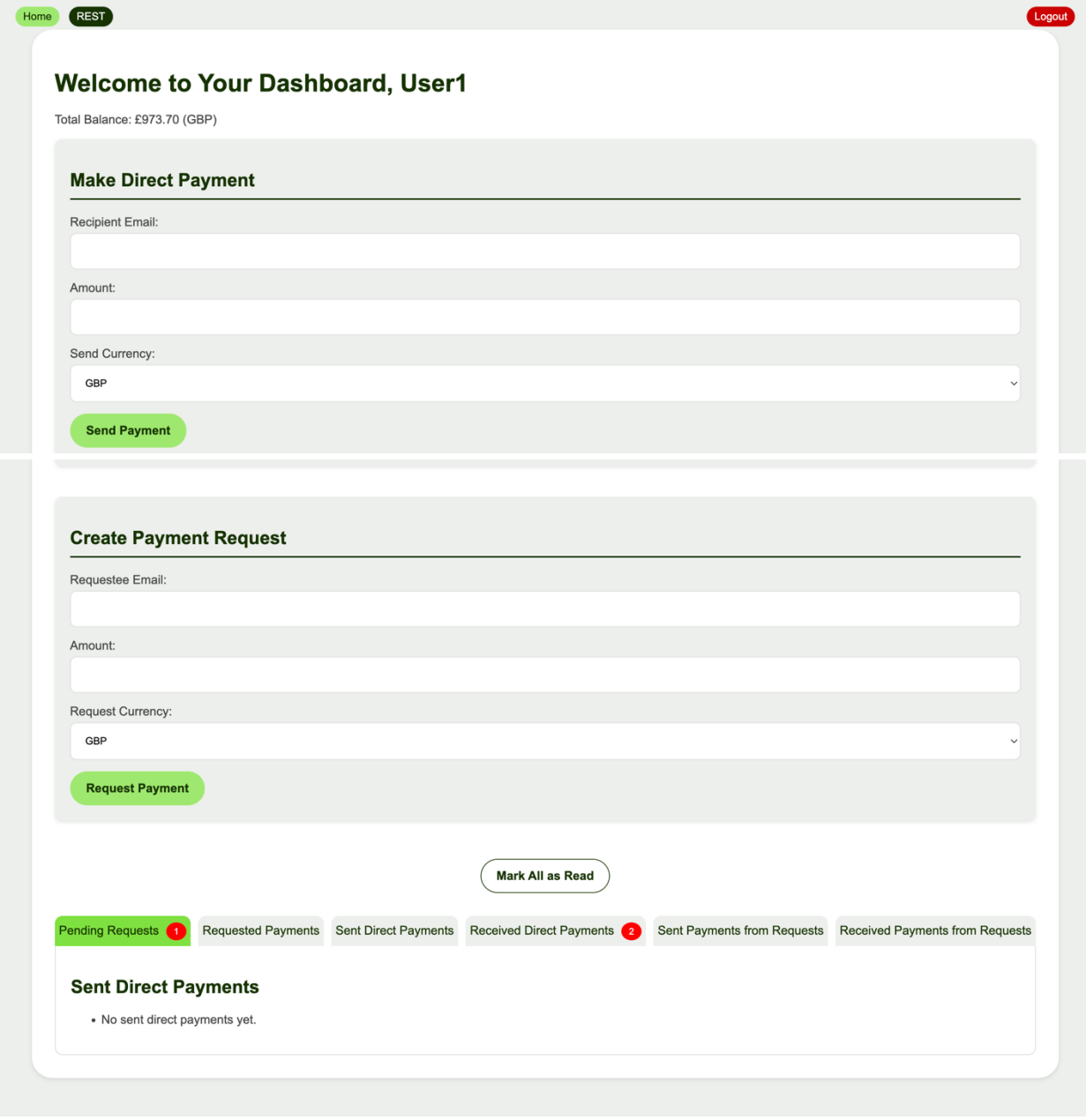


Figure 4. User dashboard.

5. Direct Payments

First section of the dashboard is a "**Make Direct Payment"** (section as seen in **Figure 5)**, which feature allows users to send money instantly to other registered users. From the **dashboard**, users enter the **recipient's email, and payment amount** before confirming the transfer. The system verifies **sufficient funds** in the sender's account and ensures the recipient exists before completing the transaction. If approved, the payment is processed within a **single Django transaction**, maintaining **ACID compliance**. Users receive an **on-screen confirmation** of the transaction and can track sent payments in their transaction history. Users can send the amount in the **desired currency**. In case the sent amount is in different currency for the sender or recipient account's currency, proper currency conversions will be carried out using the REST Currency conversion and deducted and added from the sender and recipients accounts respectively.

The message notification for payment sent with the amount sent and deduction from sender's account will be shown on the top of the dashboard on clicking the "Send Payment' button, to confirm the payment has been sent. The details of the sent payment can then be seen in the notification area at the bottom of the dashboard, this detail can be seen any time later as well. These transactions will be shown in the notification panel at the bottom of the dashboard. for sender the details will be shown in "Sent Direct Payments" tabs and receiver will see the received amount in the "received direct payments" tab.

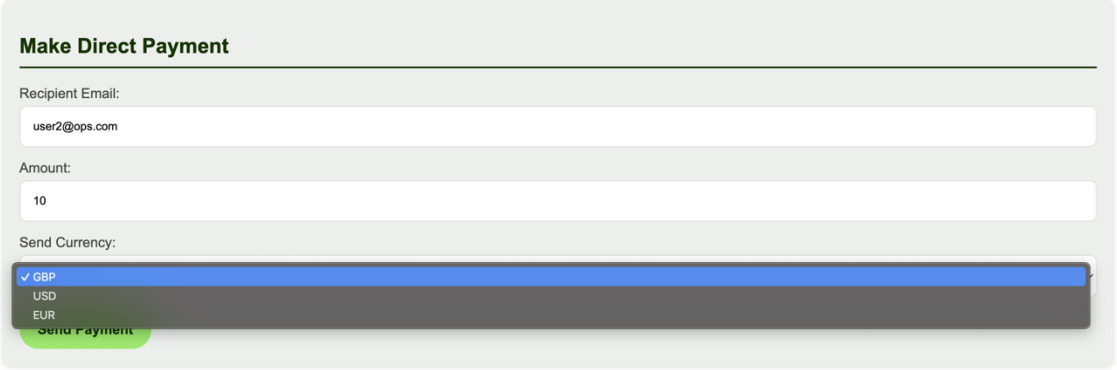


Figure 5. Make direct payment.

6. Payment Requests

Second section of the dashboard is a **"Create Payment Request"** section (as seen in **Figure** 6) which enables users to request funds from another registered user. Users input the **recipient's email, requested amount, and currency type** within the dashboard interface. The system generates a **payment request notification** (as seen in **Figure** 7**)** for the recipient, allowing them to either **accept or reject** the request. If accepted, the amount is transferred from the requester's account, updating both users' transaction histories. This feature provides a seamless way for users to manage financial interactions while ensuring all requests are tracked within the system.

The message notification for payment requested will be shown on the top of the dashboard on clicking the **"Requested Payment"** button, to confirm the payment request has been sent. The details of the sent payment request can then be seen in the notification area at the bottom of the dashboard, this detail can be seen any time later as well. For user requesting the details of request will be shown in **"Requested Payments"** tabs along with its status either pending, approved or rejected and if the payment requested is accepted the amount received will be shown under **"Received Payments from Request"** tab. On the requestee's side details of the new request will be shown under the pending request tab with the option to either accept or reject the payment request. After when the requestee accepts the payment the details of the amount send will be shown under the tab **"Sent Payments Form the Requests".** In case of requestee rejects the payment, the user who initiated will be notified with the statis of payment rejected under the tab **"Requested Payments".**

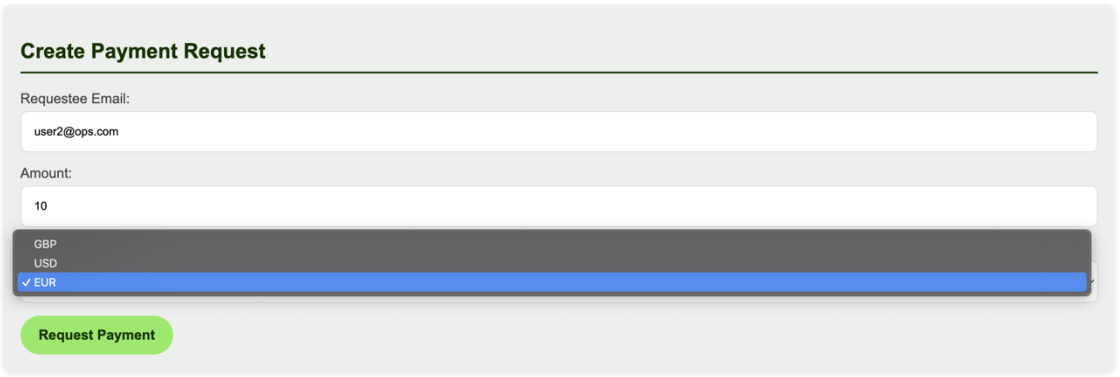


Figure 6. Create payment request.

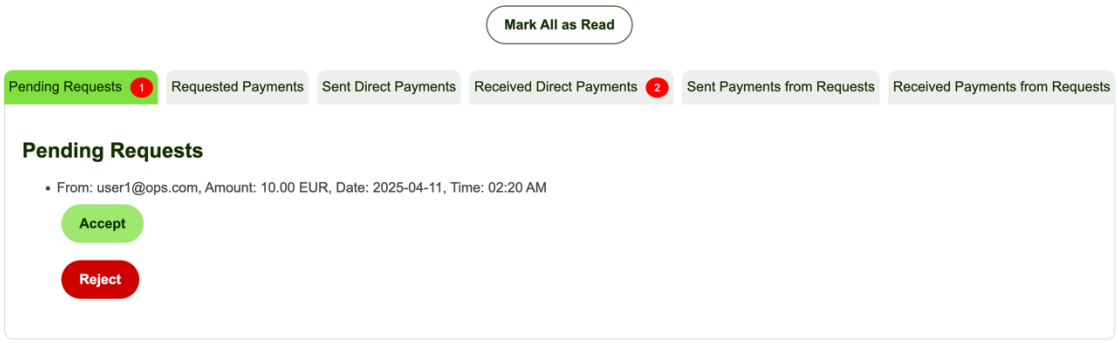


Figure 7. Pending notification.

7. REST Conversion API

The RESTful API for currency conversion in this project provides a dynamic way to exchange amounts between different currencies—GBP, USD, and EUR—using predefined exchange rates. Implemented within the payapp module, this API operates as a Django-based endpoint that responds exclusively to GET requests. The core functionality is structured around the /conversion/{currency1}/{currency2}/{amount}/ URL format, where users specify the source currency, target currency, and amount for

conversion. Upon receiving a request, the API retrieves the relevant exchange rate from a hardcoded dictionary and applies the appropriate calculation to return the converted amount. If either currency is unsupported, the service responds with an error message and an appropriate HTTP status code. This RESTful design ensures seamless integration with the payment processing system, allowing all financial transactions to reference real-time currency exchange values while maintaining simplicity and efficiency within the backend architecture. **Figure 8** below is the screenshot of the implemented RESTful API service.

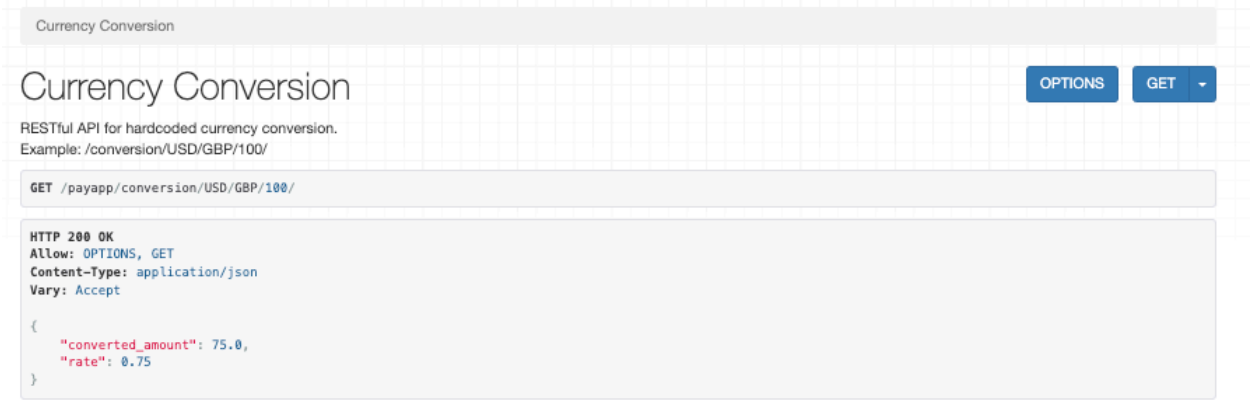


Figure 8. RESTful currency conversion.

8. Apache Thrift Base Time Stamping

The Thrift-based timestamp service in this project is designed to provide precise transaction timestamps by interfacing with a lightweight remote service running on the same server. Implemented using Apache Thrift**,** this service listens on port 10000 and responds to requests for the current system date and time. When a transaction is initiated—whether a direct payment or an accepted payment request, the Django application sends a request to the Thrift service, which returns the exact timestamp at that moment. This ensures that all financial operations are accurately logged**,** maintaining a consistent timeline for transactions**.** Using Thrift makes the service efficient and scalable**,** allowing future extensions such as external logging or multi-server synchronization. By integrating this timestamp mechanism, the application guarantees data integrity**,** making it easier to track payments while enhancing security and auditability within the system. Time stamping example can be seen in the **Figure 9** below.

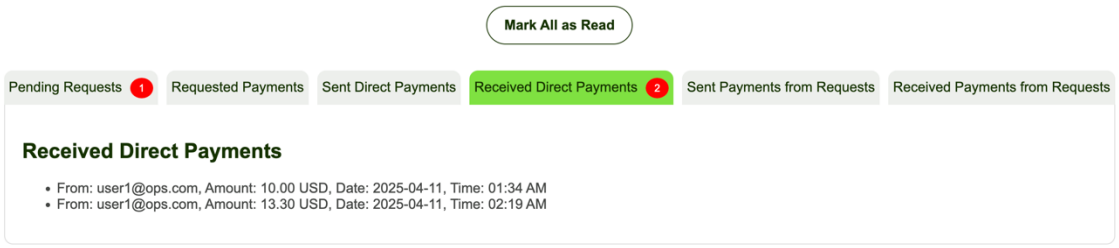


Figure 9. Time stamping

9. Admin Features

The administrator functionalities in this payment system provide complete access to user accounts and transactions, ensuring secure management of financial activities. Admins have access to a dedicated **admin dashboard** (as seen in **Figure 11**)**,** where they can view all user accounts, check balances, and track transactions in real-time. Unlike regular users, administrators can also **register new superusers** (as seen in **Figure 12**)**,** maintaining oversight of the platform's operations. Access to admin pages is strictly restricted**,** ensuring that only authorized personnel can make administrative modifications (as seen in **Figure 10)**, where entering a simple user account restricted the entry by an error message "you are not an admin". Additionally, admins can see user account, their balances and **all transactions.** This structured implementation ensures that the payment system remains transparent, secure, and properly managed**,** giving admins full visibility and control over financial interactions within the platform.

Admin login can be accessed from the top right corner button on the **user login page** (as seen in **Figure 3).** Same way user login page can be accessed from admin login page using the button at top right corner of the admin login box.

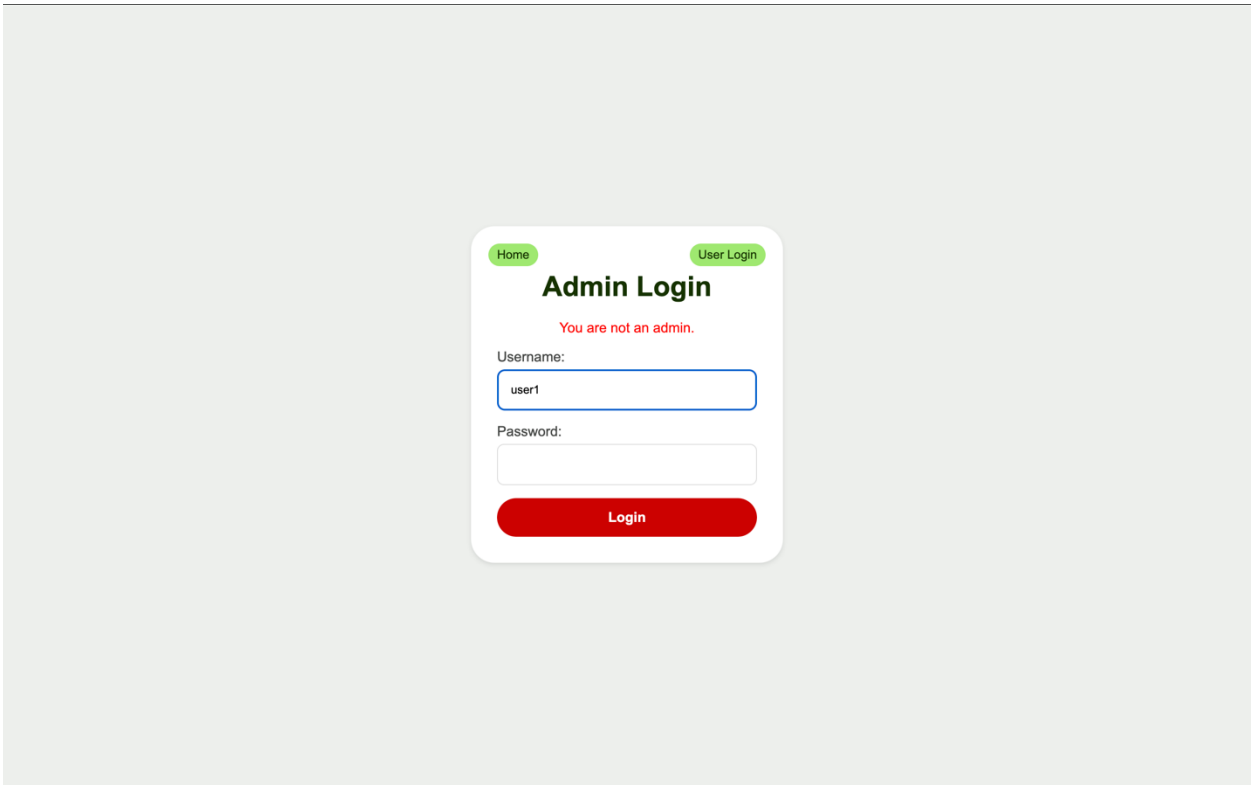


Figure 10. Admin login page.

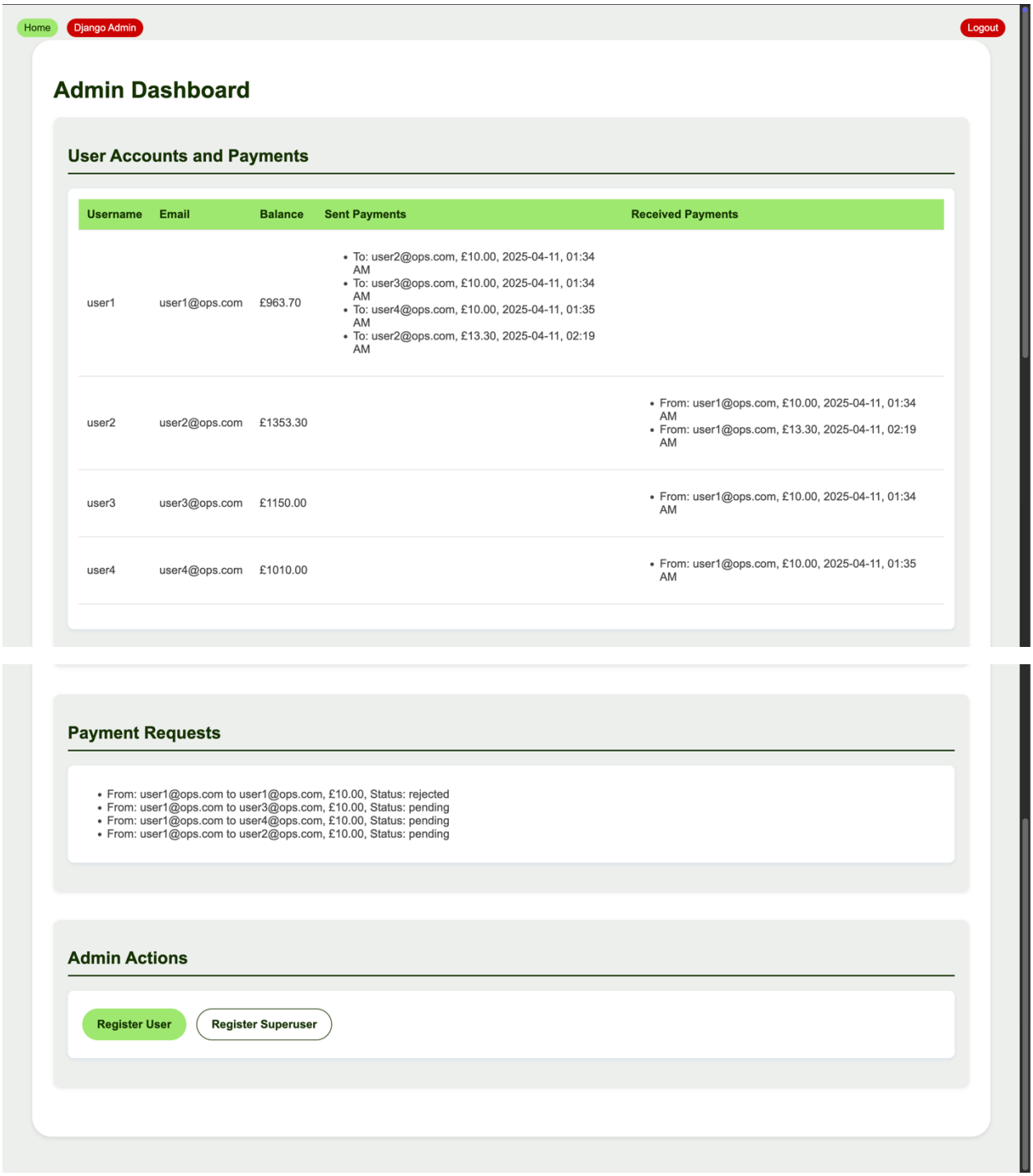


Figure 11. Admin dashboard.

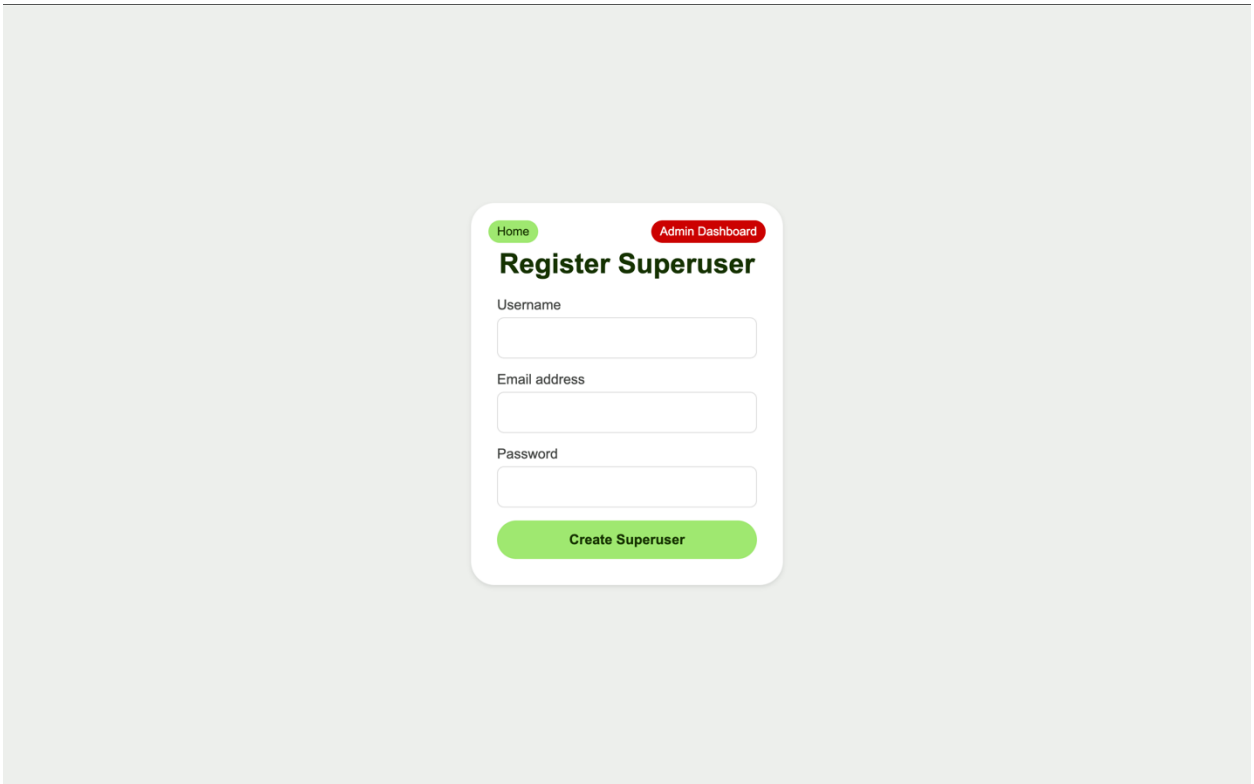


Figure 12. Register super user.

10. Security Measures

Security is a critical component of this web-based payment system, ensuring that user transactions remain protected, private, and tamper-proof**.** To achieve this, we implemented **HTTPS encryption** (as seen in **Figure 14),** securing all data exchanges between the client and server to prevent interception or unauthorized access. The authentication system ensures that only **registered users can access their accounts**, with role-based restrictions distinguishing **regular users from administrators** (as seen in **Figure 10**)**.** Login credentials are securely stored using **password hashing**, preventing exposure of raw passwords. Additionally, Django's built-in security mechanisms, such as **Cross-Site Request Forgery (CSRF) protection**, **SQL injection prevention**, and **Clickjacking defense**, safeguard the application against potential cyber threats. Access control measures ensure that **non-admin users cannot view or modify sensitive administrative data**, maintaining proper isolation between different account types. These security implementations collectively reinforce the integrity and reliability of the system, ensuring safe user interactions

11. Cloud Deployment

The AWS EC2 deployment of this project ensures that the web-based payment system runs efficiently in a cloud-based environment**,** making it accessible beyond a local development setup. By hosting the Django application on Amazon EC2**,** we achieved scalability, reliability, and security**,** allowing users to interact with the platform seamlessly. Additionally, we secured communication using HTTPS to protect sensitive user transactions and credentials. This deployment structure ensures that the system can handle multiple concurrent users**,** offering a professional and production-ready setup. The inclusion of AWS cloud services in this project demonstrates a real-world approach to hosting a financial application, reinforcing security and availability while enabling potential future scaling.

12. Deployment Process

- Created an **AWS EC2 virtual machine** running Ubuntu to host the Django application.
- Configured necessary dependencies, including **Python and Django, REST, Apache Thrift. etc.**
- Set up a **virtual environment** to manage dependencies.
- Enabled **SSL/TLS encryption** to enforce secure communication.
- Applied AWS security groups to allow only **necessary ports (80, 443, and 22 for SSH)**.
- Restricted unauthorized access to ensure **secure transactions and user authentication**
- Made adjustments to **setting.py/allowed hosts**, and added the public server for deployment.
- Started **Thrift service** to insure timestamping works on the web app.
- Started the **https server**.

Screenshots provided below demonstrate: Deployment commands executed on the AWS EC2 console (as seen in **Figure 13)** live application running on AWS with a visible URL(as seen in Fig**ure 14**), successful user interactions on the cloud-hosted platform(as seen in **Figure 15**).

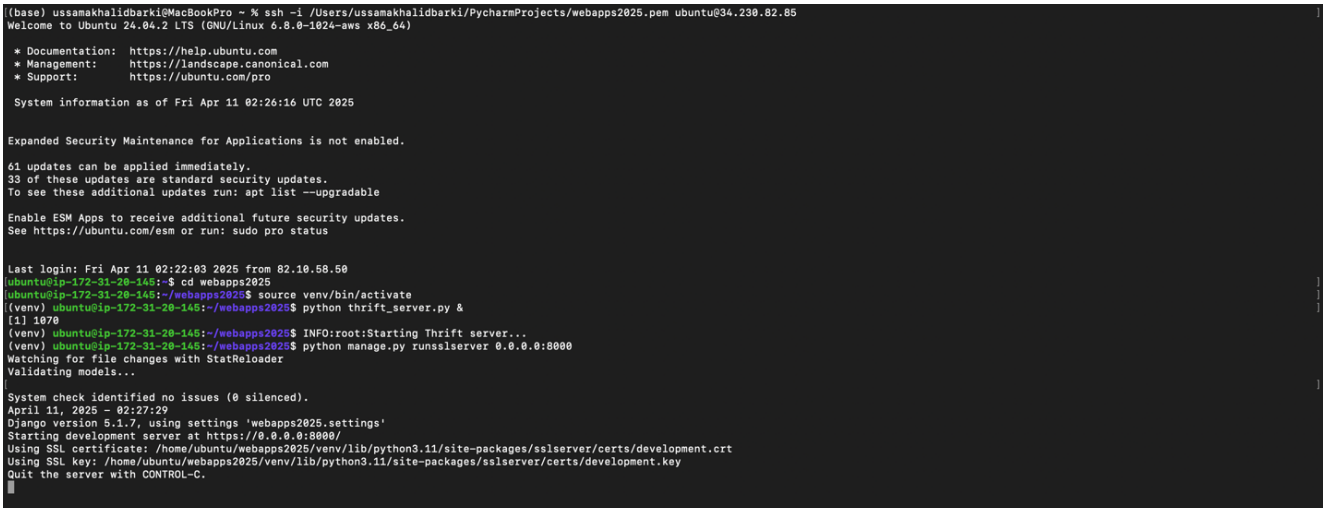


Figure 13. AWS deployment commands.

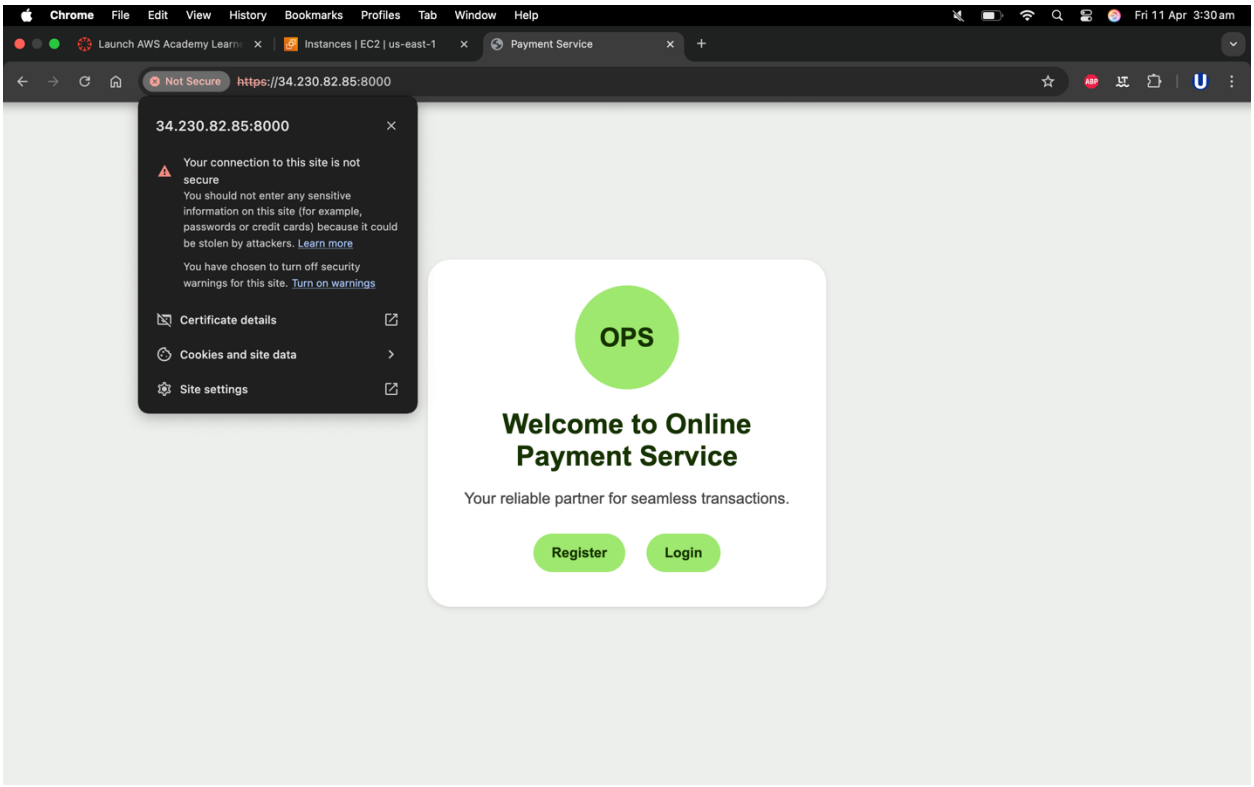


Figure 14. Public IP home page

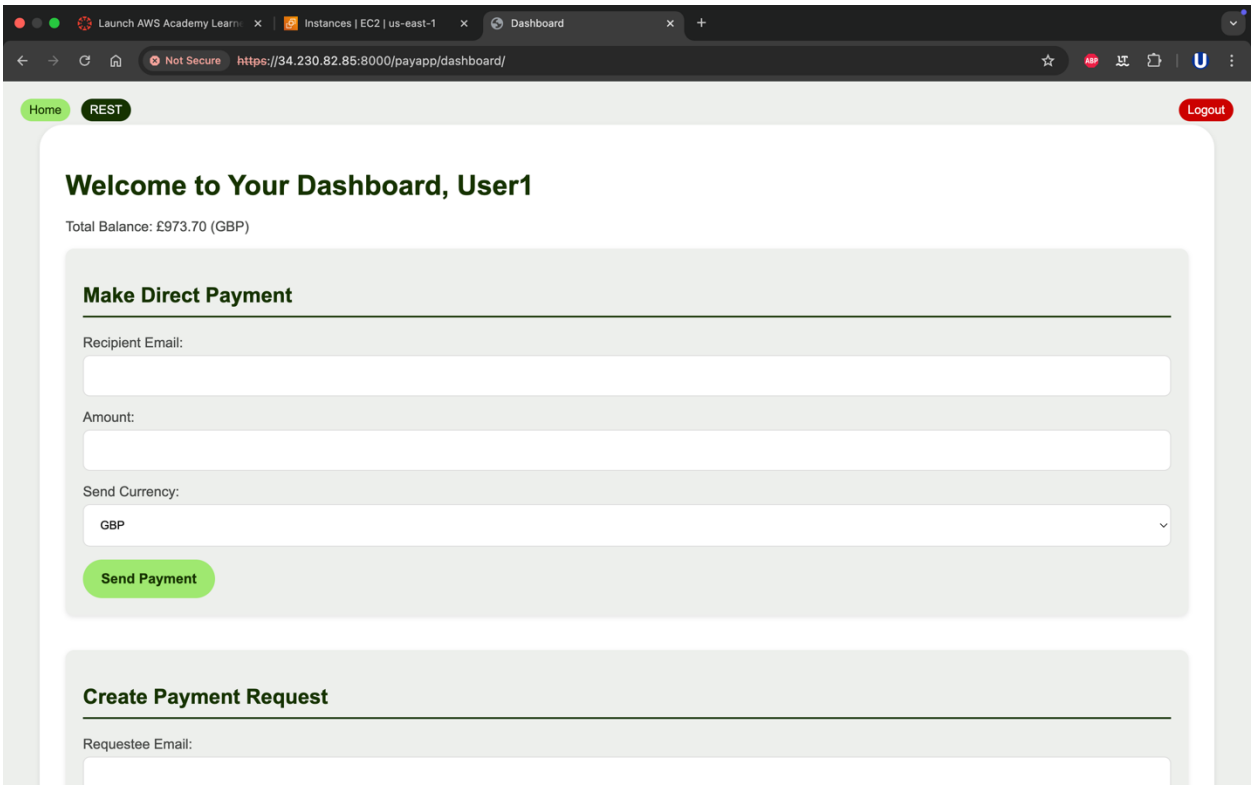


Figure 15. Public IP user dashboard page.